

C++ FILE I/O — PROPER EXAM-FORMAT THEORY ANSWERS

Descriptive university-exam answers extracted from the supplied past questions • Program-writing parts excluded

How these notes are written: Each topic is written in the form you can reproduce in an exam: definition/introduction → explanation → important details → syntax/example/diagram where required → conclusion or memory line.

1. What do you mean by Stream? Explain.

Definition: A stream is a sequence or flow of data between a program and an input/output source or destination. The program does not normally communicate with an external device or file by treating the entire source as one single operation; instead, data is transferred through a stream. Thus, a stream provides an interface between the program and the input/output device.

When data moves from an input device such as a keyboard or from a file into the program, it is called an **input stream**. When data moves from the program to an output device such as a monitor or to a file, it is called an **output stream**.

Example: In C++, `cin` is commonly used for standard input and `cout` is commonly used for standard output. Similarly, file streams are used when the source or destination is a file.

In short: A stream can be understood as a controlled flow of data between the program and an I/O source or destination.

2. What is a File Stream?

Definition: A file stream is a stream that establishes communication between a C++ program and a file stored on secondary storage. It provides facilities for transferring data from a file to the program or from the program to a file.

C++ provides file-stream classes through the `<fstream>` header file. Depending on the required operation, a program can open a file for reading, writing, or both reading and writing.

Therefore, file streams are used when data has to be stored permanently and retrieved later, rather than merely displayed temporarily on the console.

3. Discuss the Classes for File Stream with a Suitable Block Diagram.

C++ provides a set of stream classes for performing input and output operations. For file handling, the most important classes are `ifstream`, `ofstream`, and `fstream`. These classes are available through the `<fstream>` header file.

Class	Description
<code>ifstream</code>	Input file stream. It is used mainly to read data from a file into the program.
<code>ofstream</code>	Output file stream. It is used mainly to write data from the program into a file.
<code>fstream</code>	File stream capable of supporting both input and output operations.

Block diagram:

FILE STREAM CLASSES

```
|
+-----+-----+
| | |
```

```

ifstream ofstream fstream
| | |
Reading Writing Reading +
from file to file Writing

```

Thus, **ifstream** is associated with file input, **ofstream** with file output, and **fstream** can perform both. This classification makes file operations easier to organize according to the direction of data transfer.

4. Discuss the Stream Class Hierarchy.

The C++ I/O system is organized as a hierarchy of classes. The base facilities are provided by `ios_base`, while `basic_ios` provides common stream-state and stream-related facilities. Input and output functionality is then represented by input and output stream classes.

```

ios_base
|
basic_ios
+-----+
| |
istream ostream
| |
ifstream ofstream
+-----+-----+
|
iostream
|
fstream

```

ios_base: Provides basic stream facilities, including formatting-related information and stream state support.

basic_ios: Provides common functionality shared by input and output streams, including state management.

istream: Represents input streams.

ostream: Represents output streams.

iostream: Supports both input and output.

ifstream/ofstream/fstream: Extend stream facilities for file input/output.

Conclusion: The hierarchy allows C++ to provide common stream behavior while specialized classes provide the particular input, output, and file-handling operations needed by a program.

5. How can a File be Opened in C++? Explain with Syntax and Example.

Before performing input or output on a file, the file must be associated with a file-stream object. In C++, a file can be opened in two common ways: by supplying the filename to the stream object's constructor or by creating the stream object first and then calling its `open()` member function.

Method 1: Opening through constructor

```

ifstream fin("student.dat");
ofstream fout("student.dat");

```

In this method, the file is opened at the time the stream object is created. The type of stream indicates the intended direction of operation.

Method 2: Opening using `open()`

```

stream_object.open("filename", mode);

```

Example: `ifstream fin; fin.open("student.dat", ios::in);`

The `open()` function takes the filename and, when required, an opening mode. After the required operations are completed, the file should be closed using `close()`.

General sequence: Open the file → check whether opening succeeded → perform input/output → close the file.

6. What are the Different File Modes Used in C++?

File modes specify how a file is to be opened and how the stream should behave during file operations. They are represented by flags from the `ios` class and can be combined using the bitwise OR operator `|`.

Mode	Exam description
<code>ios::in</code>	Opens a file for input. Data is read from the file into the program.
<code>ios::out</code>	Opens a file for output. Data is written from the program to the file.
<code>ios::app</code>	Places each output operation at the end of the existing file, so new data is appended.
<code>ios::ate</code>	Opens the file and initially positions the file pointer at the end. The pointer may subsequently be moved.
<code>ios::trunc</code>	When used for output, existing file contents are discarded and the file is treated as a new/empty file.
<code>ios::binary</code>	Opens the file in binary mode rather than normal text mode.

Example of combining modes: `ios::in | ios::out` opens a stream for both reading and writing.

Note: A filename such as `student.dat` does not by itself mean that the file must be binary. Binary mode should be specified when binary representation is actually intended.

7. Explain How End-of-File (EOF) Condition is Handled in C++.

EOF stands for **End Of File**. It occurs when an input operation attempts to read beyond the available data in a file. A stream maintains status information so that a program can determine whether the end of the file or another error has occurred.

The `eof()` member function can be used to test whether the stream's EOF state has been set. However, when processing records, it is generally better to test the success of the actual input operation because EOF is normally recognized as a result of an attempted read.

Typical syntax: `if (fin.eof()) { /* end of file */ }`

Recommended record-reading idea: use the input/read operation itself as the condition, so the loop continues only when a complete read succeeds.

Exam conclusion: EOF is handled by checking the stream state or, preferably for record processing, by checking whether the next input operation succeeds.

8. How Can You Test Stream Errors? Explain.

During input/output, a stream may enter different error states. C++ provides member functions to test these states. These functions are useful because a program should not assume that every input, output, or file-opening operation has succeeded.

Function	Explanation
<code>good()</code>	Returns true when the stream has no error state set.
<code>eof()</code>	Returns true when the end-of-file state has been reached.
<code>fail()</code>	Returns true when an input/output operation fails, including many format-related input failures.
<code>bad()</code>	Returns true when a serious input/output error occurs.

For example, after opening a file, `if (!fin)` or `if (fin.fail())` can be used to determine whether the stream failed to open or has entered a failure state.

Remember: `good` = no error, `eof` = end reached, `fail` = operation failed, `bad` = serious I/O error.

9. Explain the Different Errors Encountered During File Operation.

File operations can fail for several reasons. The stream state functions help a program detect these conditions rather than continuing as if the operation were successful.

Error / condition	Description
File cannot be opened	The requested file may not exist, the path may be wrong, or access may not be available.
Read failure	The requested data cannot be extracted successfully, for example because the input does not match the expected format.
Write failure	The program is unable to successfully write the requested data.
EOF	No more data is available for reading from the file.
Invalid stream state	A previous failure may leave the stream in a state in which later operations do not work normally.
Access/permission problem	The operating system may prevent reading or writing because of permissions or other access restrictions.
Serious I/O error	A severe input/output problem can set the stream's bad state.

10. Explain Different Types of File Access.

File access refers to the method by which records or data stored in a file are reached. The two basic approaches commonly discussed are sequential access and random (direct) access.

1. Sequential access: In sequential access, data is processed in a particular order, normally from the beginning toward the end of the file. To reach a later record, the program generally passes through the preceding data. This method is simple and suitable when records are processed one after another.

2. Random or direct access: In random access, the current file position can be moved to a required location. Therefore, a program can reach a particular portion of a file without processing every preceding record. C++ supports movement of file pointers using functions such as `seekg()` and `seekp()`.

Difference: Sequential access follows the order of records, whereas random access allows the program to move directly to a required file position.

11. What are Different File Access Pointers? Explain.

A file access pointer represents the current position in a file from which input is read or at which output is written. C++ mainly uses two logical file positions: the **get pointer** and the **put pointer**.

Get pointer: It indicates the position from which the next input operation will read data. Functions associated with it include `seekg()` for moving the pointer and `tellg()` for obtaining its current position.

Put pointer: It indicates the position at which the next output operation will write data. Functions associated with it include `seekp()` for moving the pointer and `tellp()` for obtaining its current position.

Function	Purpose
<code>seekg()</code>	Moves the get/read pointer.
<code>tellg()</code>	Returns the current get/read position.
<code>seekp()</code>	Moves the put/write pointer.
<code>tellp()</code>	Returns the current put/write position.

Memory: g = get/read, p = put/write; seek = move, tell = report the current position.

12. Write Down the Different Techniques for Formatting I/O Stream.

I/O stream formatting means controlling the appearance and representation of data during input/output. Proper formatting is especially useful when displaying tables, reports, bills, marksheets, or numerical results. C++ provides manipulators, stream member functions, and formatting flags for this purpose.

Common formatting techniques include: setting field width, controlling decimal precision, choosing fixed or scientific notation, selecting left or right alignment, and selecting a fill character for unused field positions.

Technique	Explanation
setw(n)	Sets the minimum width of the next output field to n characters.
setprecision(n)	Controls the precision of floating-point output; with fixed notation, n commonly represents digits after the decimal point.
setfill(ch)	Specifies the character used to fill unused positions in a field.
fixed	Displays floating-point values in fixed-point notation.
scientific	Displays floating-point values in scientific notation.
left / right	Controls alignment of output within the specified field width.

Most of these manipulators are provided by `<iomanip>`. They are used with the insertion operator `<<` to make output organized and readable.

13. List the Features Used in Formatting the Output. Explain Each.

Output formatting features determine how values are arranged and represented when they are sent to an output stream. The most useful features in examination and practical programs are width, precision, fill character, alignment, and floating-point notation.

Width: Width determines the minimum number of character positions reserved for an output field. `setw( )` is commonly used to specify it.

Precision: Precision controls the number of significant digits or, when combined with `fixed`, the number of digits after the decimal point. It is controlled using `setprecision( )`.

Fill character: If the data does not occupy the entire field width, the unused positions can be filled with a chosen character using `setfill( )`.

Alignment: Data can be aligned to the left or right side of the field using `left` and `right`.

Floating-point format: `fixed` displays numbers in fixed notation, while `scientific` displays them in scientific notation.

14. Write Short Notes on File Access Pointers and Their Manipulators.

File access pointers maintain the current location used for file input and output. A program may need to move to another record or determine where it currently is. C++ provides pointer-manipulating functions for these operations.

seekg() moves the get pointer, while **seekp()** moves the put pointer. **tellg()** reports the current get-pointer position, while **tellp()** reports the current put-pointer position.

These functions are particularly useful for random/direct file access, where the program needs to jump to a particular location instead of processing the entire file sequentially.

15. Binary File Concept — Theory Note Relevant to the Past Questions

A binary file stores data in a form that represents the underlying bytes of the stored information rather than arranging the data as ordinary human-readable text. Binary file operations are useful when a program wants to

store and retrieve records in a compact representation.

In C++, binary mode is requested using `ios::binary`. Record-oriented binary file handling commonly uses stream functions such as `write()` and `read()`.

Important exam clarification: The extension `.dat` does not itself mean that a file is binary. The file is binary when the program opens/handles it in binary mode or the question explicitly requires a binary file.

16. How to Answer the Repeated Past-Question Variations

Several of the supplied questions repeat the same theory concepts with slightly different wording. The following combinations let you prepare one complete answer rather than memorizing separate answers.

Past-question wording	Prepare this answer
Classes for file stream operator + block diagram	Section 3: file-stream classes + diagram; add Section 4 if 'hierarchy' is asked.
How a file can be opened + syntax/example	Section 5.
Different file modes	Section 6.
EOF condition	Section 7.
Test stream errors	Section 8.
Errors during file operation	Section 9.
Different file access / access pointers	Sections 10 and 11.
Formatting I/O stream / output formatting	Sections 12 and 13.
File access pointers and manipulators	Section 14.
What is stream / file stream	Sections 1 and 2.
Stream class hierarchy + ios flags	Sections 4 and 6.

17. Last-Minute Memory Map

STREAM → flow of data

FILE STREAM → program ↔ file

ifstream → read

ofstream → write

fstream → read + write

OPEN → constructor / open()

MODES → in, out, app, ate, trunc, binary

EOF → end of available file input

ERRORS → good, eof, fail, bad

ACCESS → sequential / random

POINTERS → get/put; seekg/seekp; tellg/tellp

FORMATTING → width, precision, fill, alignment, fixed/scientific

Exam-writing rule: If the question says *Discuss* or *Explain*, do not write only a list. Start with a definition, explain the concept in 1–2 paragraphs, then use a table/diagram/examples to support the explanation, and finish with a short concluding statement.